

# 인공지능시스템

## AI System

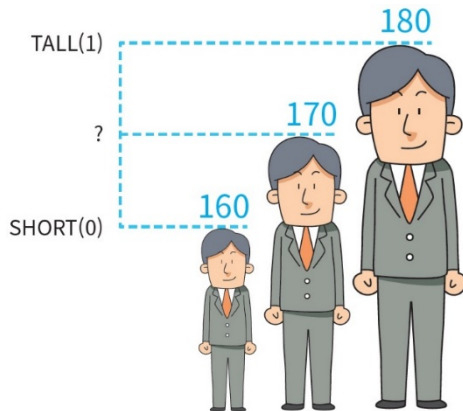
int**Ro**-Lab.

청주대학교 무인항공기학과  
김상현 교수 ([www.intro-lab.com](http://www.intro-lab.com))

## ◆ 퍼지 논리란?

- 퍼지(Fuzzy) 정의: Not clear, distinct, or precise; blurred → “흐릿하다”는 뜻.
- 퍼지 논리(Fuzzy Logic): 명확하게 정의될 수 없는 지식을 표현하는 방법.
- 인간이 사용하는 모호한 단어나 표현을 논리적으로 정의할 수 있도록 도와 줌.

## ◆ 인간의 모호한 표현 예시: 키가 크다, 작다. → **명확한 규칙(기준)이 없음.**



## ◆ 이진 논리

- 참과 거짓, 1 혹은 0, 흑과 백  $\rightarrow$  0 혹은 1이라는 두 개의 진리값만이 존재
- LED가 켜졌다 혹은 꺼졌다.

## ◆ 퍼지 논리

- 0.0에서 1.0까지의 연속적인 진리값을 가짐.
- LED가 밝다 혹은 어둡다 등의 애매한 표현을 정의.

## ◆ 퍼지 논리 역사

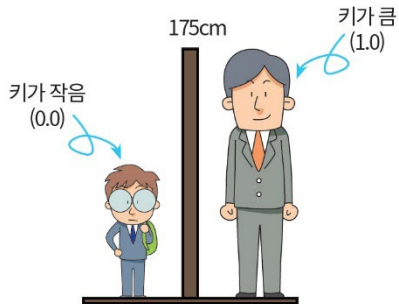
- 1965년 로트피 자데(Lotfi A. Zadeh)에 의해 퍼지 집합에 관한 이론이 처음 제시 됨.
- 퍼지 명제나 규칙을 다루기 위한 퍼지 논리로 발전 됨.



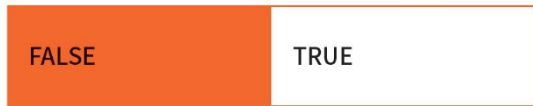
로트피 애스커 자데(Lotfi Asker Zadeh,  
[1921년 2월 4일](#) ~ [2017년 9월 6일](#))

# 퍼지 논리

## ◆ 이진 논리와 퍼지 논리 비교 예시



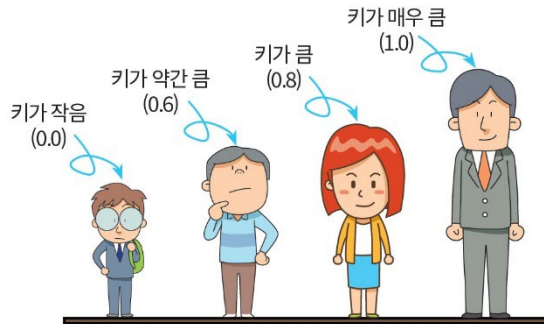
이진 논리



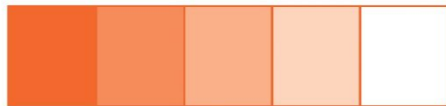
0

1

이진 논리



퍼지 논리



0.0

0.2

0.4

0.6

0.8

1.0

퍼지 논리

## 퍼지 논리

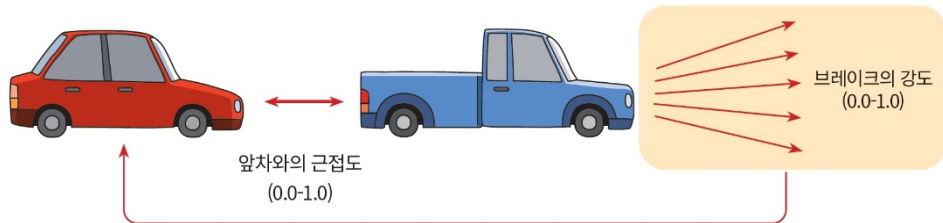
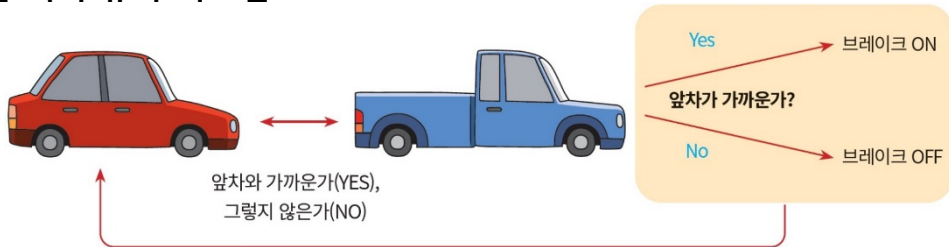
- ◆ 과거 퍼지라는 단어는 인공지능 기술을 대표하는 단어였음.



# 퍼지 논리

## ◆ 퍼지 논리를 사용할 수 있는 분야

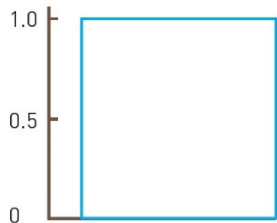
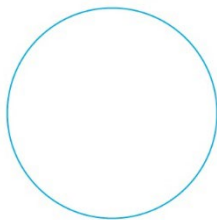
### ● 자동차 차간 거리 유지 시스템



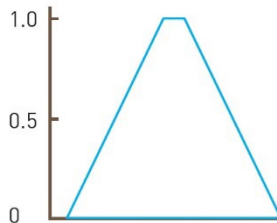
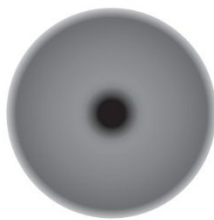
## ◆ 퍼지 논리와 집합

- 명제 논리 == 기존 집합 (크리스프 집합) → 집합에 소속되면 1, 아니면 0
- 퍼지 논리 == 퍼지 집합 → 집합 중심에 근접하면 1, 멀리 있으면 0

크리스프 집합

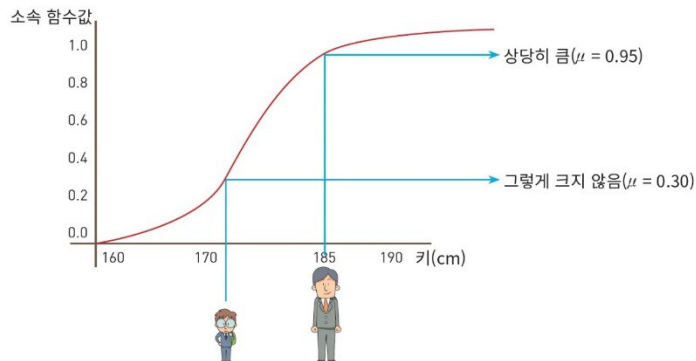
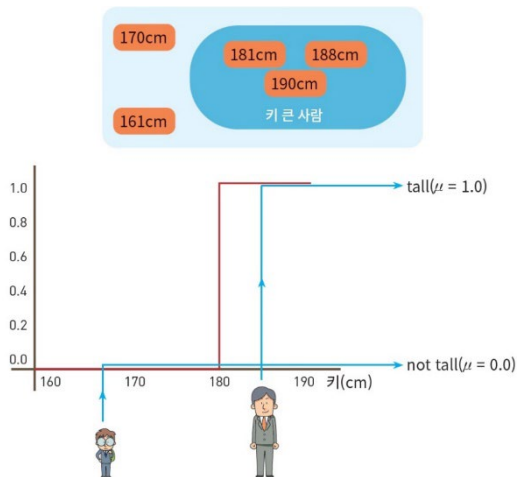


퍼지 집합



## ◆ 소속 함수(Membership Function)

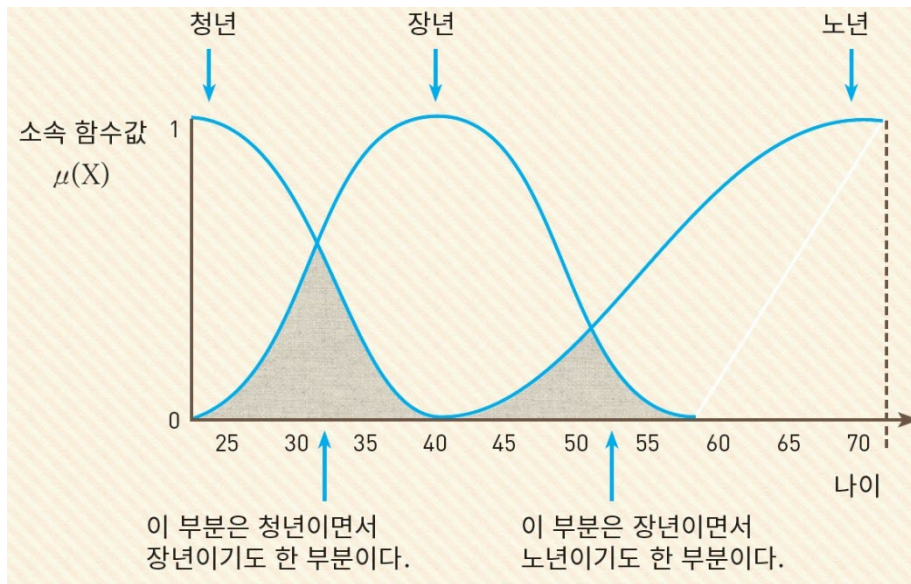
- 명제 논리: 원소가 집합에 소속되어 있으면 소속 함수 결과 값이 1, 아니면 0
- 퍼지 논리: 원소가 집합에 소속되어 있는 정도에 따라 소속 함수 값이 0.0~1.0 사이 값을 가짐.



# 퍼지 논리

## ◆ 퍼지 집합 예시

- 청년, 장년, 노년



## ◆ 명제 논리 연산자

| A | B | $\min(A, B)$ |
|---|---|--------------|
| 0 | 0 | 0            |
| 0 | 1 | 0            |
| 1 | 0 | 0            |
| 1 | 1 | 1            |

AND

| A | B | $\max(A, B)$ |
|---|---|--------------|
| 0 | 0 | 0            |
| 0 | 1 | 1            |
| 1 | 0 | 1            |
| 1 | 1 | 1            |

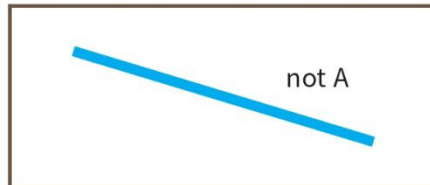
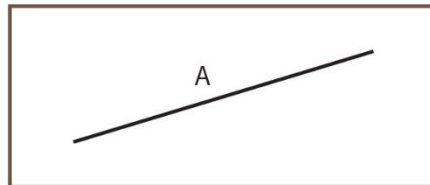
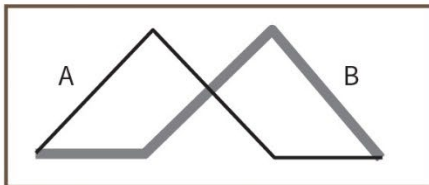
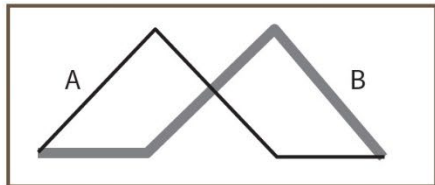
OR

| A | $1 - A$ |
|---|---------|
| 0 | 1       |
| 1 | 0       |

NOT

# 퍼지 논리

## ◆ 퍼지 논리 연산자



# 퍼지 논리

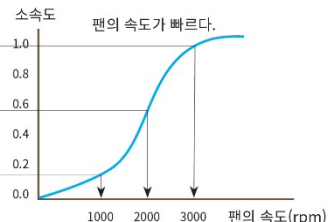
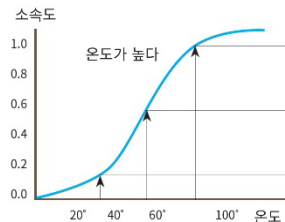
## ◆ 퍼지 추론

### ● 기존의 추론

- 규칙 #1: IF 온도가 높다. THEN 팬의 속도를 증가시킨다.
- 사실 #1: 온도가 약간 높다.
- -----
- 추론된 사실: ???

### ● 퍼지 추론

- 규칙 #1: IF 온도가 높다. THEN 팬의 속도를 빠르게 한다.
- 사실 #1: 온도가 약간 높다.
- -----
- 사실 #2: 팬의 속도를 약간 빠르게 한다.



## ◆ GAME: Trackmania 2020



**UBISOFT Presents**

## ◆ Training an unbeatable AI in Trackmania



# 불확실성

## ◆ 고전적인 논리

- 추론을 위한 규칙의 전제 조건이나 결론이 100% 참 혹은 100% 거짓이어야 함.

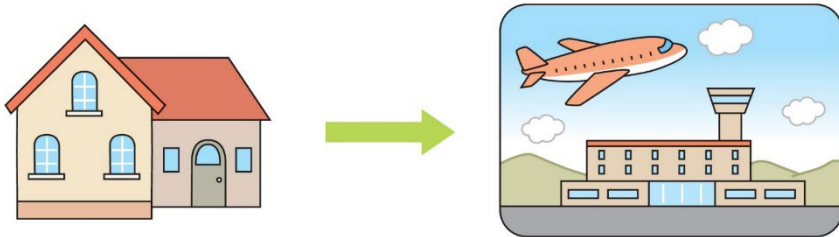
## ◆ 현실 세계에서 100% 확실한 정보나 지식을 얻는다는 것은 상당히 어려움.

- 따라서 우리가 사용하는 정보나 지식은 항상 어느 정도 불확실함.

## ◆ 불확실성의 예

### ● 집에서 공항까지 가는 문제

- 중간에 자동차가 고장날 수 도 있고, 차량 정체가 있을 수 있음.
- 공항이 몹시 혼잡하여 수속에 시간이 많이 걸릴 수도 있음.



## ◆ 불확실성의 예

- 내과에서 환자를 진단하는 전문가 시스템
  - 규칙: 감기이다 → 열이 있다.
  - 규칙을 100% 확신할 수 없다.



### 감기증상



두통



쇠약감



열



기침



근육통



가슴통증

## ◆ 불확실성은 왜 발생하는가?

- 데이터의 불확실성: 측정 장치의 센서 오차나 오류 존재 가능.
- 지식의 불완전성: 전문가의 지식도 완벽하지 않음.
- 정보 획득의 불완전성: 문제를 해결하기 위한 모든 정보를 수집하는 것이 불가능.
- 부정확한 언어: 인간의 언어(자연어)에는 모호성이 있음.

## ◆ 불확실성을 다루는 방법

- 수학적인 도구로 확률 이론(Probability Theory)이 있음.

## ◆ 대표적인 확률 이론

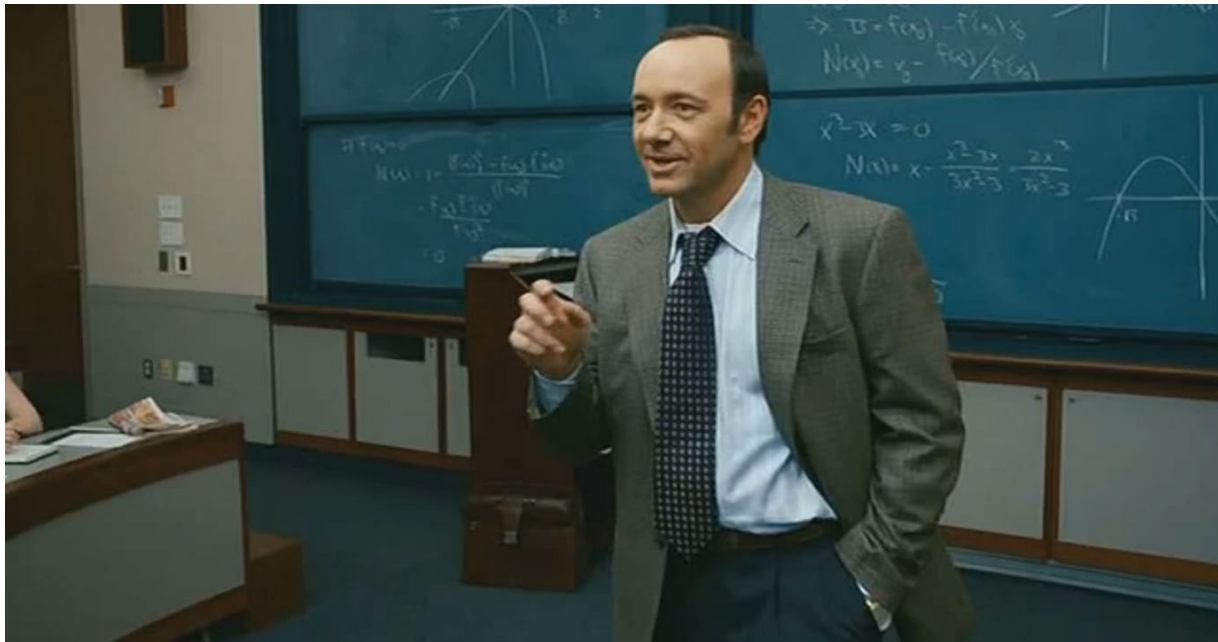
- 베이즈 추론(Bayesian Reasoning)

## 확률 이론

- ◆ 확률적인 추론(Probabilistic Reasoning)에서는 명제의 진리값이 0(거짓) 또는 1(참)이 아니고 0.0에서 1.0 사이의 값을 가짐.
- ◆ 확률적인 추론에서는 가장 확률이 높은 결론이 선택됨.



## ◆ 몬티홀 문제



◆ 베이지 정리



## ◆ 베이즈 정리

### ● 사전 확률과 사후 확률

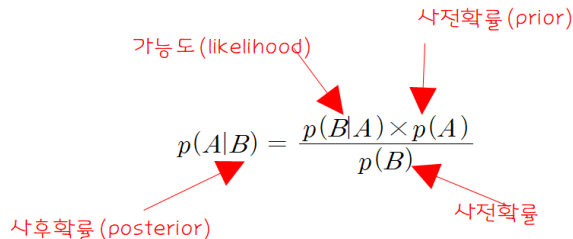
$$p(A|B) = \frac{A \text{와 } B \text{가 동시에 발생할 경우의 수}}{B \text{가 발생할 경우의 수}}$$

$$p(A|B) = \frac{p(A \cap B)}{p(B)}$$

$$p(B|A) = \frac{p(A \cap B)}{p(A)}$$

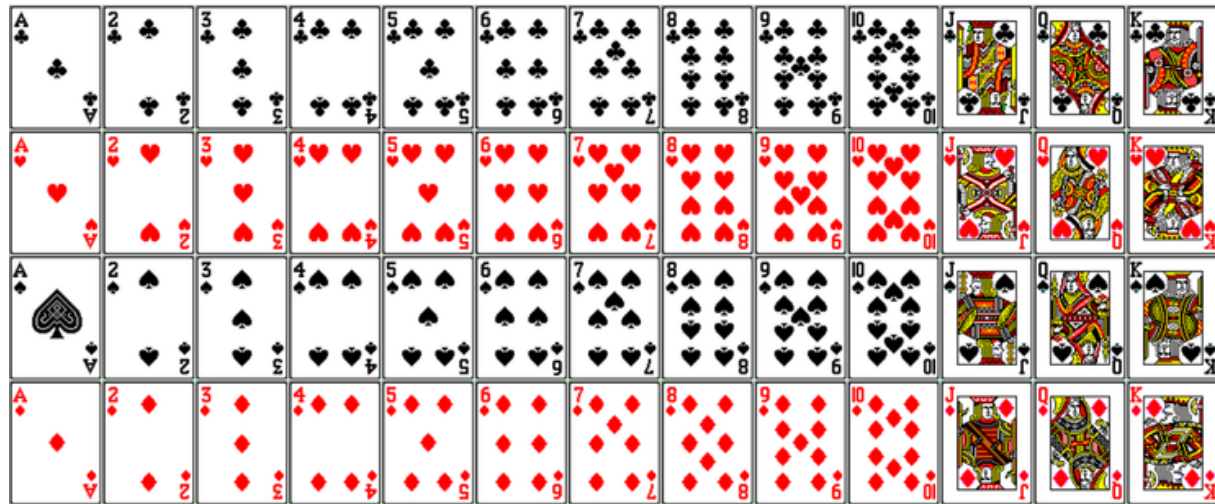
$$p(A \cap B) = p(B|A) \times p(A)$$

$$p(A|B) = \frac{p(B|A) \times p(A)}{p(B)}$$



## ◆ 베イズ 정리

- 예시: 카드 게임
- 우리가 한 장의 카드를 가지고 있는데 카드에 얼굴이 그려져 있다는 것만 알고 있다. 이 상황에서 이 카드가 킹(king)일 확률을 추론해보자. 즉  $P(\text{King} \mid \text{Face})$  확률을 계산해보는 것이다.



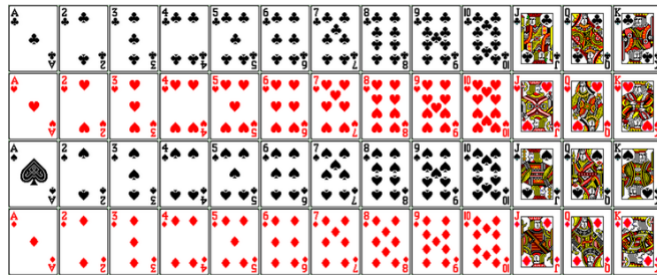
## ◆ 베이지 정리

● 예시: 카드 게임

$$p(A|B) = \frac{p(B|A) \times p(A)}{p(B)}$$

$$P(King|Face) = \frac{P(Face|King)P(King)}{P(Face)}$$

$$= \frac{1 \times \frac{1}{13}}{\frac{3}{13}} = \frac{1}{3}$$



## ◆ 베이즈 정리

### ● 예시: 충치

➤ 치통(B)이 있을 때, 충치(A)일 확률

|     | 치통   | ¬치통  |
|-----|------|------|
| 충치  | 0.04 | 0.06 |
| ¬충치 | 0.01 | 0.89 |

$$p(A|B) = \frac{p(B|A) \times p(A)}{p(B)}$$

$$P(\text{충치} | \text{치통}) = \frac{P(\text{충치} \wedge \text{치통})}{P(\text{치통})} = \frac{0.04}{(0.04 + 0.01)} = 0.8$$

## ◆ 파이썬

- Thonny IDE 다운로드

<https://thonny.org>



The screenshot shows the Thonny website's download page. At the top, the word "Thonny" is displayed in a large, bold, black serif font. To its right is a green download icon and the text "Download version 5.0.0 for Windows • Mac • Linux". Below this, a white box with a light gray border contains the heading "Official downloads for Windows". Under this heading, there are four download options, each with a description and a link. The third option, "Portable variant with Python 3.14 for Intel or AMD computers, also usable on Arm", is highlighted with a red rectangular border. The link for this option is "thonny-py38-5.0.0-windows-portable-x64.zip (36 MB)". Below the highlighted option, there is a section for "Re-using an existing Python installation (for advanced users)" with the command "pip install thonny". At the very bottom of the screenshot, a small portion of a code editor is visible, showing a function definition: "def fact(n):".

**Thonny**

Download version [5.0.0](#) for Windows • Mac • Linux

**Official downloads for Windows**

**Installer with Python 3.14 for Intel or AMD computers**, requires Windows 10 or 11  
[thonny-5.0.0-x64.exe \(24 MB\)](#) 🖱️ *recommended for you*

**Installer with Python 3.14 for Arm (e.g. Snapdragon) computers**  
[thonny-5.0.0-arm64.exe \(23 MB\)](#)

**Portable variant with Python 3.14 for Intel or AMD computers**, also usable on Arm  
[thonny-py38-5.0.0-windows-portable-x64.zip \(36 MB\)](#)

**Re-using an existing Python installation** (for advanced users)  
pip install thonny

if n == 0: def fact(n):

## ◆ 베イズ 정리 코드

<https://colorscripter.com/>

```
# A: 충치, B: 치통

# P(A): 충치일 사전 확률,  $P(A) = P(A \text{ and } B) + P(A \text{ and } (\text{not } B))$ 
P_A = 0.04 + 0.06

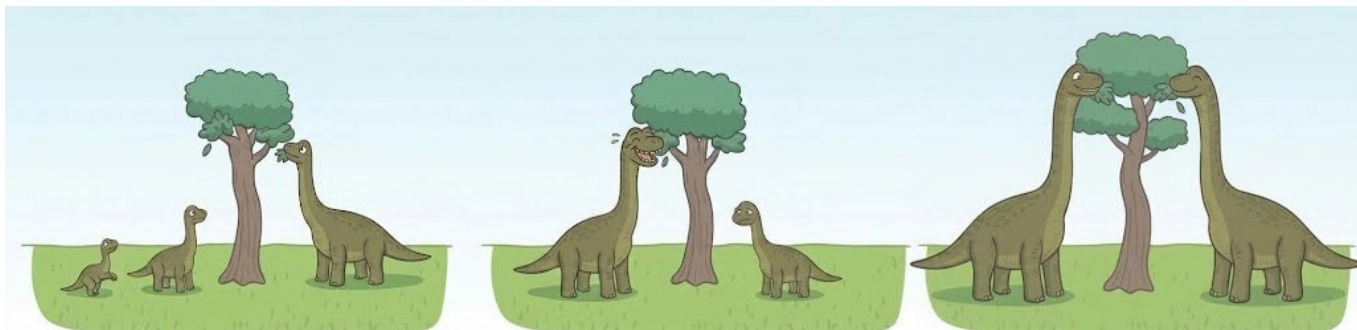
# P(B): 치통이 있을 사전 확률,  $P(B) = P(A \text{ and } B) + P((\text{not } A) \text{ and } B)$ 
P_B = 0.04 + 0.01

# P(B|A): 충치일 때 치통이 있을 확률, 가능성도,  $P(B|A) = P(A \text{ and } B) / P(A)$ 
P_B_given_A = 0.04 / P_A

# 베イズ 정리를 이용하여 사후 확률  $P(A/B)$  계산
P_A_given_B = P_B_given_A * P_A / P_B
print(P_A_given_B)
```

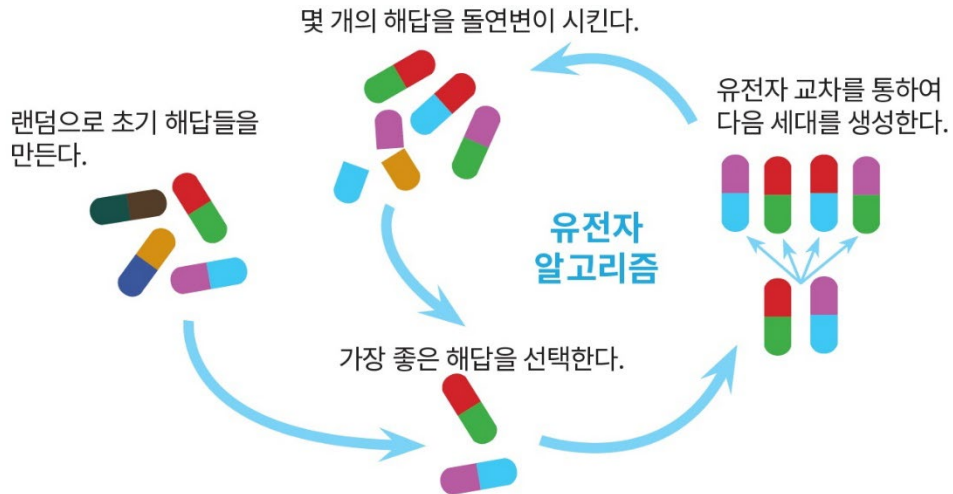
## 유전자 알고리즘

- ◆ 유전자 알고리즘은 생물체의 염색체가 유전되는 현상에서 영감을 얻은 최적화 알고리즘.
- ◆ 적자생존 원칙에 의해 교차, 돌연변이, 도태 등의 과정을 반영.
- ◆ 즉, 주어진 환경에 적합한 유전자만이 살아남는 자연계의 현상을 알고리즘으로 만든 것.



# 유전자 알고리즘

## ◆ 유전자 알고리즘 수행 과정



# 유전자 알고리즘

## ◆ 적합도(Fitness)

- 현재의 염색체가 얼마나 문제를 잘 해결하고 있는지에 대한 점수

## ◆ 평가함수(Evaluation Function)

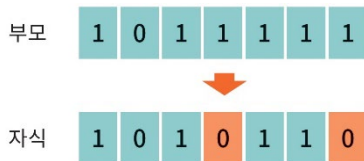
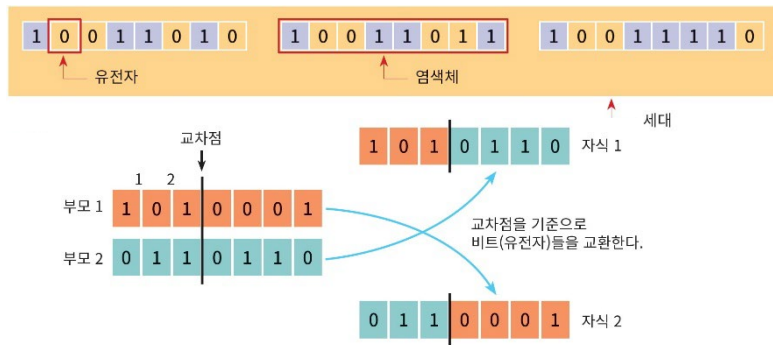
- 적합도를 계산하는 함수

## ◆ 교차(Crossover)

- 부모 염색체의 일부가 교환되는 것

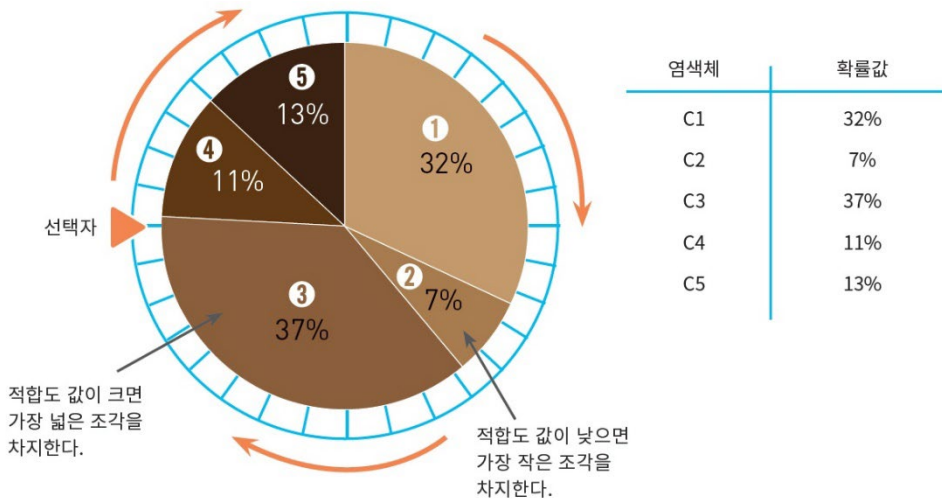
## ◆ 돌연변이(Mutation)

- 염색체에서 랜덤한 위치의 유전자가 값이 변동되는 것

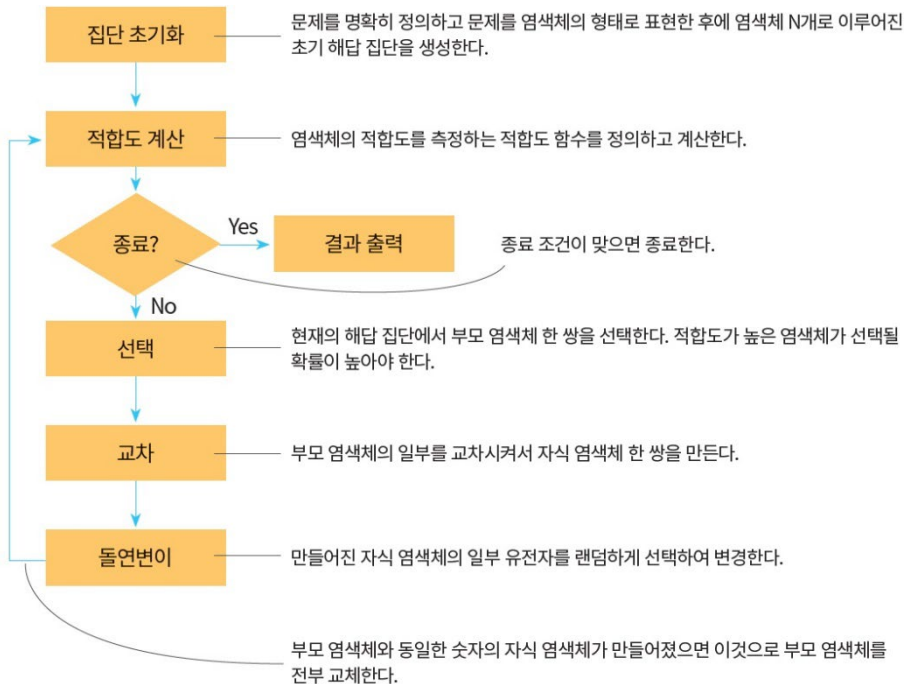


## ◆ 선택 연산자

- 유전자 알고리즘은 재산을 위하여 우수한 성능(높은 적합도)을 가지는 부모 염색체 2개를 선택
- 이때 사용되는 알고리즘이 룰렛 휠 선택 → 즉, 높은 적합도이면 높은 선택 확률



## ◆ 전체 과정



## ◆ 유전자 알고리즘 시뮬레이션

[https://youtu.be/Yr\\_nRnqeDp0](https://youtu.be/Yr_nRnqeDp0)

